

Task Collection Models and Commit Breakdown

Prepared for GREED

Project: Diddys-task-manager

Date: 2026-05-12

1) Shared Contract: `ITaskCollection<T>`

All four collections implement the same interface. This gives your app one common API while letting each data structure behave differently under the hood.

```
public interface ITaskCollection<T>
{
    void Add(T item);
    bool Remove(T item);
    T FindById(int id);
    ITaskCollection<T> Filter(Func<T, bool> predicate);
    void Sort(Comparison<T> comparison);
    int Count { get; }
    bool Dirty { get; set; }
    R Reduce<R>(R initial, Func<R, T, R> accumulator);
    IMyIterator<T> GetIterator();
    IMyIterator<T> GetEnumerator();
    T[] ToArray();
    void Clear();
}
```

Important concept: every model tracks `Dirty` so services/repositories can know whether data changed and should be persisted.

2) ArrayTaskCollection<T>

Storage: plain array `T[] items`.

- **Add:** creates a new array (+1 size), copies old values, appends item.
- **Remove:** finds index, allocates smaller array, copies all except removed element.
- **FindById:** linear scan + reflection (`GetProperty("Id")`).
- **Sort:** uses `Array.Sort` and sets `Dirty=true`.
- **Iterator:** index-based custom iterator over the array.

Complexities: Add $O(n)$, Remove $O(n)$, Find $O(n)$, Sort $O(n \log n)$.

Best fit: simple implementation and predictable behavior for small/medium lists.

3) LinkedListTaskCollection<T>

Storage: custom singly linked nodes (`Data` + `Next`).

- **Add:** walks to last node and links new node.
- **Remove:** tracks `prev / current` and relinks around removed node.
- **FindById:** node-by-node scan + reflection for `Id`.
- **Sort:** converts to array, sorts, clears list, re-adds in sorted order.
- **Iterator:** custom linked-list iterator with `Reset()` support.

Complexities: Add $O(n)$ (because no tail pointer), Remove $O(n)$, Find $O(n)$.

Best fit: when pointer-style structure is desired and index access is not needed.

4) HashMapTaskCollection

Storage: hash table with bucket array and chained entries.

- **Key:** `TaskItem.Id`.
- **Bucket index:** `Math.Abs(key.GetHashCode()) % _buckets.Length`.
- **Add:** updates existing key or inserts new entry at chain head.
- **Remove:** unlinks entry from collision chain.
- **FindById:** direct bucket lookup then short chain traversal.
- **Sort:** intentionally unsupported (`NotSupportedException`).

Complexities (average): Add $O(1)$, Remove $O(1)$, Find $O(1)$. Worst case can degrade to $O(n)$ with heavy collisions.

Best fit: high-speed lookup by task id.

5) BinarySearchTreeTaskCollection

Storage: binary search tree ordered by `TaskItem.Id`.

- **Add:** recursive insert to left/right subtree by id.
- **Remove:** handles 3 classic cases: leaf, one child, two children (successor replacement).
- **FindById:** tree walk left/right by id comparison.
- **ToArray:** in-order traversal returns tasks in ascending id order.
- **Sort:** unsupported because the tree is inherently ordered by key.

Complexities (average): Add $O(\log n)$, Remove $O(\log n)$, Find $O(\log n)$. Worst case $O(n)$ if tree becomes skewed.

Best fit: ordered retrieval with decent lookup/update performance.

6) Performance Comparison

Operation	Array	Linked List	HashMap	BST
Add	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$ avg
Remove	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$ avg
FindById	$O(n)$	$O(n)$	$O(1)$ avg	$O(\log n)$ avg
Sort	Supported	Supported	Not Supported	Not Supported (ordered by key)

7) Your Commits (GREED) - What They Show

From your recent commit history on `main`:

```
a681ed5 task dependency and readme
05b2f77 Task dependency feature
da8bae9 unit test feature 4 en contribution
f3a1bc5 Unit tests made
ecd3db2 implemented binary search tree
c749d9d Remove generated bin/ and obj/ build artifacts
...
```

Interpretation:

- You introduced advanced data structure support with the BST implementation.
- You strengthened project quality with unit test-focused commits.
- You improved repository hygiene by removing generated artifacts.
- You tied features together with task dependency updates and documentation improvements.

8) Code-Level Strengths in Your Work

- Consistent interface across all models keeps service code clean and swappable.
- Custom iterator abstraction is implemented across each structure.
- `Dirty` flag is consistently updated after mutating operations.
- Tree and hash structures prioritize lookup speed over generic flexibility.
- Error handling for missing IDs is explicit through exceptions.

Final Note

GREED, your repository shows clear progression from base features to advanced structures and testing. The four models are not duplicates; they are practical demonstrations of data-structure tradeoffs in real task management behavior.